PROJECT 9

Telecommunications Network Operations, Customer Experience & Service Assurance Analytics Platform

Five-Day Capstone: Implementation Roadmap, Plain-English Guidance and Trackers

Item	Detail
Domain	Telecommunications and Network Service Assurance
Duration / Team	5 working days / 5 trainees
Data	7 source files (6 CSV + 1 JSON), 6,900 records
Platforms	IDMC, Snowflake, SnowSQL, Snowpark Python, Streamlit, Git
Architecture	Bronze – Silver – Gold medallion
Document version	Refined edition, 30 October 2026

How this document is organised

- **Part A – Understand:** the project in plain English, a coverage matrix that maps every point of the specification to a day and an owner, open questions and team roles.
- **Part B – Day-wise roadmap:** Day 1 to Day 5, step by step, with owners, outputs, evidence and an end-of-day exit check.
- **Part C – How to implement (layman guidance):** concepts explained simply, plus ready-to-adapt SQL, Python and Streamlit starters for every layer.
- **Part D – Trackers:** testing matrix, reconciliation template, acceptance checklist, deliverables checklist, common pitfalls and the demo script.

> **Important:** Where the specification is silent (for example the exact value lists, which row carries which defect, or who may see unmasked data), this document states an **assumption** in Section A4. Confirm these with your trainer on Day 1 and write the final decision in your design document.

PART A – UNDERSTAND THE PROJECT

A1. The project in plain English

A telephone company has seven separate "notebooks" kept by different teams: who the customers are, which services they bought, which network equipment (towers, routers, switches) exists, how much each customer used, which network faults happened, which customers complained, and the raw alarm messages from the network-monitoring system. Today nobody can easily answer: **"When this tower degraded, which services broke, which customers were affected, and how quickly did we fix it?"**

Your job is to collect those seven notebooks, clean them, load them into Snowflake in three progressively cleaner layers (Bronze → Silver → Gold), connect them with trustworthy keys, calculate service-assurance KPIs, score each network element as Low / Medium / High risk, secure customer data, and show everything in a Streamlit dashboard.

> **In plain words:** Think of a water-treatment plant. **Bronze** is the raw river water (kept exactly as received). **Silver** is filtered water (correct types, no duplicates, valid relationships). **Gold** is bottled water ready to drink (dimensions, facts and marts that dashboards can use directly). IDMC is the pre-filter that sorts good, doubtful and unusable water before it enters the plant.

A2. How the seven files connect

Understanding these relationships is the single most important design step. Write them on a whiteboard on Day 1.

Parent (one)	Child (many)	Join key	Notes
customers	subscriptions	customer_id	One customer can hold several subscriptions (900 customers, 1,400 subscriptions).
subscriptions	usage_transactions	subscription_id	usage also carries customer_id; check it agrees with the subscription's customer.
network_elements	usage_transactions	network_element_id	Which element served the usage.
network_elements	network_elements	parent_element_id → network_element_id	Self-reference = hierarchy (for example site → controller → core). Needs a recursive query for the full path.
network_elements	network_incidents	primary_element_id → network_element_id	Element where the incident originated.
network_incidents	trouble_tickets	network_incident_id	Link may be empty for tickets not caused by a network incident; confirm in profiling.
customers / subscriptions	trouble_tickets	customer_id, subscription_id	Who complained and about which service.
network_elements	network_events (JSON)	network_element_id	Raw alarms; services affected are inside the JSON array affected_services.

Correlation logic (objective: "correlate NMS events, incidents, services and tickets"): an event is linked to an incident when it comes from the same element (or one listed in related_elements) and its timestamp falls between the incident start and end (optionally with a small tolerance window such as 30 minutes before start). Tickets join to incidents by network_incident_id; services join through the affected_services array to subscriptions by service_type or service_identifier (confirm which in profiling).

A3. Requirement coverage matrix (nothing missed)

Every section of the specification is traced below to where it is delivered. Use this as your master checklist.

§	Requirement	Delivered in	Day / Owner
1	Governed customer–subscription–network view; reduced 7-file scope	Gold model + secure views	D3 / M1
2	Trusted customer-subscription-network analytical view	DIM_CUSTOMER, DIM_SUBSCRIPTION, DIM_NETWORK_ELEMENT + facts	D3 / M1
2	Measure voice, SMS and data usage	FACT_USAGE, MART_USAGE_SUMMARY	D3 / M3
2	Incident frequency, duration, severity, customer impact	FACT_NETWORK_INCIDENT, MART_NETWORK_RELIABILITY	D3 / M1
2	Ticket resolution and SLA compliance	FACT_TROUBLE_TICKET, MART_TICKET_SLA	D3 / M5
2	Correlate events, incidents, services, tickets	FACT_NETWORK_EVENT + correlation view (A2)	D3 / M5
2, 11	Transparent network-risk score with Snowpark	Snowpark script → MART_NETWORK_RISK	D4 / M5
2	Secure network and customer-experience dashboards	Streamlit app on secure views	D5 / M5
3.1	Customer, subscription, element hierarchy, usage, incidents, tickets, JSON events	Bronze/Silver/Gold for all 7 files	D2–D3 / all
3.1	One usage incremental pipeline	Stream + Task on usage	D4 / M4
3.1	RBAC and customer-data masking	3 roles, masking policies, secure views	D4 / M4
3.2	Out of scope: account/plan/device masters, billing, complaints, portability, offers, real-time OSS, Kafka/MQTT, network control, Cortex AI, MLOps	Do NOT build. account_id, plan_id, device_id stay as plain attributes on DIM_SUBSCRIPTION (no dimension)	All
4	IDMC, Snowflake, Snowpark, Streamlit, SQL/Python/Git/Snowsight	Used as per Part B	All
5	Layers: Sources → IDMC → Stage → Bronze → Silver → Gold → Consumption → Control	Architecture diagram + mapping document	D1 / M1
6, 7	7 files, 6,900 records, every column in correct order	Header validation, Bronze column-order test, reconciliation	D1–D2 / M2, M3
8	Exact DQ counts: 8 nulls, 3 duplicates, 3 anomalies, 2 outliers, 4 date formats	DQ register (D1), IDMC routing (D2), DQ report (D2/D5)	D1–D2 / M2
8.1	3 sequence anomalies (subscription, incident, ticket)	Reject route + quarantine table	D2 / M2
8.2	2 outliers (1,000,000 MB; 500,000 customers)	Warning-and-retain, flagged, capped in scoring	D2–D4 / M2, M5
8.3	Routing: clean / standardize / warning / duplicate / reject	IDMC routers → 4 output files per source	D2 / M2
9	4 dimensions, 4 facts, 4 marts	Gold DDL and load scripts	D3 / M1, M3
10	10 KPI families	KPI catalogue (D1) + validation queries (D3)	D1, D3 / M1
11	Component scores, total, category, scoring timestamp	MART_NETWORK_RISK columns	D4 / M5
12	Three roles, masking, Bronze restricted, secure views, USE ROLE validation, admin roles minimised	Security scripts + screenshots	D4 / M4
13	Initial load, second file (new + corrected), idempotent replay, Gold mart refresh	Stream/Task test pack	D4 / M4
14	Five-day plan and evidence per day	Part B	All
15	11 testing areas	Part D test matrix	D3–D5 / M5

§	Requirement	Delivered in	Day / Owner
16	9 deliverable groups	Part D deliverables checklist	D5 / all
17	9 acceptance criteria	Part D acceptance checklist	D5 / all

A4. Gaps and assumptions to confirm on Day 1

The specification is precise about counts but silent on several details. Decide these early and record them; they affect your IDMC rules, Silver logic and KPIs.

#	Topic	Our working assumption	How to confirm
1	Which 4 date/timestamp formats are mixed, and in which files	Formats such as YYYY-MM-DD, DD/MM/YYYY, MM-DD-YYYY, DD-Mon-YYYY. All become ISO (YYYY-MM-DD, YYYY-MM-DD HH24:MI:SS).	Profile every date column with a pattern check. Watch dd/mm vs mm/dd ambiguity (13/02 is safe, 03/04 is not).
2	Which exact rows hold the 8 nulls, 3 duplicates, 3 anomalies, 2 outliers	Defects do not overlap (each row has at most one defect).	Build a DQ register: file, row, column, defect, expected route.
3	Are the 3 duplicates exact copies or same key with different content?	Same business key; keep the latest source_updated_ts, send the extra copy to the duplicate route.	Group by key; compare columns.
4	Does the 6,900 include duplicate and reject rows?	Yes. 6,900 = loaded + 3 duplicates + 3 rejects, so expect about 6,894 rows to reach Silver.	Count after profiling; adjust if defects overlap.
5	Are rejected and duplicate rows kept anywhere?	Yes, in QUARANTINE tables (for audit) so the 6,900 total always reconciles. They are not used in Silver or Gold.	Agree with trainer.
6	Are the 8 nulls only in optional columns?	Yes, so they route as warnings (retain). If a null hits a key or mandatory column, escalate and treat as reject.	Check null columns during profiling.
7	Allowed value lists (severity, status, priority, element_type, usage_type)	Derived from profiling; documented in a reference list.	SELECT DISTINCT per column.
8	SLA compliance definition	A ticket meets SLA when (resolve_ts − create_ts) in minutes ≤ sla_target_minutes. Open tickets are reported separately, not as breached.	Confirm with trainer.
9	Mean time to restore (MTTR)	Average of (incident_end_ts − incident_start_ts) in minutes for incidents with a valid end time.	Confirm with trainer.
10	"Open critical alarms"	Events with severity = Critical and event_status not in (Cleared/Closed), taken from network_events.	Check real status values.
11	Who sees unmasked customer data	SERVICE_ASSURANCE_ROLE sees contact and address (needs to call customers). NETWORK_ENGINEER_ROLE and TELECOM_MANAGER_ROLE see masked values. ACCOUNTADMIN is not used for viewing.	Confirm with trainer; record in the access matrix.
12	Second usage file for incremental test	Not supplied. The team creates usage_transactions_batch2.csv: about 100 new rows + about 20 corrected rows (same ID, newer source_updated_ts).	Prepare on Day 2 or early Day 4.
13	Customer history (SCD)	Latest-wins (Type 1) because only source_updated_ts is available.	Mention in design notes.
14	IDMC availability	If a licence or connectivity problem blocks IDMC, document it and use an equivalent SQL/Python cleansing step so the schedule is not lost.	Test login on Day 1.

A5. Suggested team roles (5 trainees)

Member	Primary role	Owns
M1	Team lead, architect, Gold modelling and KPIs	Architecture, grain/keys, DIM_*, marts, KPI catalogue, final presentation
M2	IDMC lead	Profiling support, cleansing, routing, clean/warning/duplicate/reject files, DQ report, second-file creation
M3	Snowflake data engineer	Environment, stages, file formats, Bronze loads, Silver, facts, performance, Time Travel/clone recovery
M4	Security, incremental and runbook owner	Roles, masking, secure views, Stream/Task, replay tests, runbook
M5	Snowpark, Streamlit and QA lead	Risk score, Streamlit app, test matrix, UAT, defect log, event-JSON flattening

> Roles are for ownership, not isolation. Pair up for reviews: every script is read by a second person before it is run on shared objects.

PART B – DAY-WISE IMPLEMENTATION ROADMAP

Each day has a goal, numbered tasks with owners, the evidence to capture (screenshots, scripts, files) and an exit check. Do not start the next day until the exit check passes; every later day depends on the previous one being correct.

B0. Five days at a glance

Day	Theme	What exists at end of day	Evidence
1	Discover and design	Profiling results, DQ register, grain/key table, KPI catalogue, architecture, source-to-target mapping, Snowflake environment ready	Profiling, design and KPI catalogue
2	Clean, route and land (IDMC → Bronze)	7 sources cleansed and routed to clean/warning/duplicate/reject files; stages, file formats, Bronze loaded; audit reconciliation	Clean/reject files, Bronze and audit evidence
3	Model (Silver → Gold)	Typed Silver; 4 dimensions, 4 facts, 4 marts; KPIs validated; full reconciliation	Silver, Gold, KPIs and reconciliation
4	Automate, secure, score	Stream/Task with second load and replay; roles and masking; Snowpark risk score; performance and recovery	Incremental, security, score and recovery evidence
5	Present and prove	Streamlit app; end-to-end tests; UAT; defects closed; runbook; final demo	Application, tests, UAT, runbook, presentation

DAY 1 – Domain Discovery, Profiling, Design and Environment

Goal: Know the data completely before touching any tool. Every decision on Days 2 to 5 comes from what you learn today.

#	Task	Owner	Output
1.1	Kick-off (60 min): read the specification together, agree the glossary (NMS, SLA, MTTR, element, incident, ticket), confirm roles (A5) and the working rules: naming standard, Git branch rules, daily 15-minute stand-up and 30-minute end-of-day review.	All	Team charter page
1.2	Create the Git repository with folders: /docs, /idmc, /snowflake/01_setup … /snowflake/09_recovery, /snowpark, /streamlit, /tests, /evidence/day1…day5.	M1	Repo with README
1.3	Create the Snowflake environment (see C2): warehouse TELCO_WH (X-Small, auto-suspend 60 s), database TELCO_DB and schemas STG, BRONZE, SILVER, GOLD, QUARANTINE, AUDIT, PIPELINE, SECURITY. Check SnowSQL connects.	M3, M4	Setup script, SnowSQL login screenshot
1.4	Test IDMC login, connections and a sample mapping run. Raise any access problems today, not on Day 2.	M2	IDMC readiness note
1.5	Profile all seven files (split: M2 customers + subscriptions; M3 network_elements + usage; M5 tickets + incidents + JSON; M1 consolidates). For each column record: row count, column order vs dictionary, null count, distinct count, min/max, date patterns, value lists, key duplicates.	M2, M3, M5	Profiling workbook (one tab per file)
1.6	Locate the controlled defects and build the DQ register: 8 nulls, 3 duplicates, 3 sequence anomalies (subscription end < start, incident end < start, ticket resolve < create), 2 outliers (1,000,000 MB usage; 500,000 impacted customers), 4 mixed date formats. Record file, row, column, defect, expected route.	All	DQ register with exact counts
1.7	Run relationship profiling: orphan checks for every join in A2 (subscriptions→customers, usage→subscriptions, usage→elements, tickets→incidents, incidents→elements, events→elements, element parent→element).	M3, M5	Orphan report
1.8	Define grain and keys for every Silver and Gold table (see C7 table). Decide surrogate vs natural keys.	M1	Grain and key table
1.9	Write the KPI catalogue: name, business meaning, formula, source tables, grain, filters, owner (see C8).	M1, M5	KPI catalogue
1.10	Draw the architecture (eight layers of §5) and write the source-to-target mapping for all 7 files across Bronze → Silver → Gold.	M1, M4	Architecture diagram, mapping sheet
1.11	Define routing rules and expected counts per file (clean, warning, duplicate, reject); this becomes the target for Day 2 reconciliation.	M2	Routing rule sheet
1.12	Resolve the open assumptions in A4 with the trainer; update the documents.	M1	Signed-off assumptions

Evidence to capture

- Profiling workbook, DQ register, orphan report
- Architecture diagram, grain/key table, mapping sheet, KPI catalogue
- Screenshot of Snowflake objects created and SnowSQL connection

Exit check

- Every one of the 6,900 records is accounted for in profiling (900 + 1,400 + 400 + 2,200 + 700 + 400 + 900).
- The DQ register totals exactly 8 / 3 / 3 / 2 / 4.

- Every team member can explain the chain customer → subscription → usage → element → incident → ticket.

DAY 2 – IDMC Cleansing, Routing, Stages and Bronze

Goal: Turn raw files into routed, audited Bronze tables. Rejects and duplicates are separated, correctable formats are standardised, nothing is silently lost.

#	Task	Owner	Output
2.1	Build one IDMC mapping per file (7 mappings): source → standardise (trim, case, ISO dates) → validate (nulls, sequences, outlier thresholds) → de-duplicate → router → four targets.	M2 (all help)	7 mappings
2.2	Implement the routing rules of §8.3: Valid → Clean; Correctable format (dates) → standardise and keep in Clean; Optional null → Warning; Duplicate → Duplicate file; Impossible sequence → Reject; Extreme but possible (outliers) → Warning and retain.	M2	Router logic evidence
2.3	Add a reason column on Warning/Duplicate/Reject outputs (for example REASON_CODE = END_BEFORE_START) so every exception is explainable.	M2	Reason codes
2.4	Handle network_events.json: validate JSON structure and the 12 documented keys, check event_id not null and duplicates, keep nested objects/arrays untouched, write clean/warning/reject JSON.	M5, M2	JSON mapping
2.5	Create an IDMC taskflow that runs all seven mappings and writes a run log. Capture a successful run.	M2	Taskflow screenshot
2.6	Compare IDMC output counts with the Day 1 expected counts. Investigate any difference before moving on.	M2, M1	Routing count comparison
2.7	Create file formats (CSV, JSON) and an internal stage with one sub-folder per source (see C5).	M3	Stage/format script
2.8	Upload files with SnowSQL PUT; confirm with LIST.	M3	PUT/LIST output
2.9	Create the Bronze tables (all VARCHAR, same column order as the dictionary, plus metadata columns) and the JSON Bronze table (one VARIANT column plus metadata). Load with COPY INTO and metadata capture.	M3, M4	Bronze DDL, COPY results
2.10	Create AUDIT tables (batch log, file log, row-count log) and QUARANTINE tables for rejects and duplicates; load the reject/duplicate files into QUARANTINE.	M4	Audit and quarantine tables
2.11	Reconcile: source file rows = clean + warning + duplicate + reject; Bronze rows = clean + warning. Save the result as the DQ and reconciliation report (draft).	M3, M2	Reconciliation report v1
2.12	Prepare the second usage file (batch 2) with new and corrected records for Day 4.	M2	usage_transactions_batch2.csv

Evidence to capture

- Clean / warning / duplicate / reject files and IDMC mapping + taskflow screenshots
- LIST @stage output, COPY INTO results, Bronze row counts
- AUDIT and QUARANTINE tables, reconciliation report v1

Exit check

- Exactly 3 duplicates, 3 rejects, 2 outliers (warning), 8 optional nulls (warning), 4 formats standardised.
- Bronze row counts match IDMC clean + warning counts for each file.
- Column order in Bronze equals the data dictionary for all seven tables.

DAY 3 – Silver and Gold: Dimensions, Facts and Marts

Goal: Produce trusted, relationship-validated Silver tables and the full Gold model, then prove the KPIs are right.

#	Task	Owner	Output
3.1	Build Silver for the six CSV sources: cast to proper types with TRY_ functions, trim/standardise, de-duplicate on the business key (latest source_updated_ts wins), add flags (is_outlier, is_warning).	M3	6 Silver tables
3.2	Flatten network_events.json into a typed Silver table: 12 columns with metric_context, alarm_context, geo_context as OBJECT and affected_services, related_elements as ARRAY; add a child table for exploded services if needed for KPIs.	M5	SILVER.NETWORK_EVENTS (+ services child)
3.3	Relationship validation in Silver: write orphan queries for every join in A2; route orphans to a SILVER exception table and document why.	M3, M5	Orphan = 0 report or exception table
3.4	Business-sequence validation again in Silver (end ≥ start for subscriptions, incidents, tickets) as a safety net.	M3	Validation queries
3.5	Gold dimensions: DIM_DATE (generated calendar), DIM_CUSTOMER, DIM_SUBSCRIPTION (keep account_id, plan_id, device_id as attributes), DIM_NETWORK_ELEMENT (include hierarchy level and path via recursive query).	M1	4 dimensions
3.6	Gold facts: FACT_USAGE, FACT_NETWORK_INCIDENT (duration_minutes), FACT_TROUBLE_TICKET (resolution_minutes, sla_met_flag), FACT_NETWORK_EVENT (flattened counts, critical flag).	M3, M5	4 facts
3.7	Gold marts: MART_USAGE_SUMMARY, MART_NETWORK_RELIABILITY, MART_TICKET_SLA; create MART_NETWORK_RISK structure (populated Day 4).	M1, M5	3 marts + risk table shell
3.8	Build the incident-event-service-ticket correlation view (A2).	M5	VW_INCIDENT_CORRELATION
3.9	Validate every KPI in §10 by calculating it two independent ways (for example Silver SQL vs Gold mart) and comparing.	All	KPI validation sheet
3.10	Full reconciliation: Source → Bronze → Silver → Gold counts and key sums (for example total data_volume_mb, total charge_amount).	M3, M1	Reconciliation report v2
3.11	Commit all scripts to Git with clear messages; peer-review each other's work.	All	Git history

Evidence to capture

- Silver and Gold DDL/load scripts, row-count screenshots
- KPI results with validation queries
- Reconciliation report v2 (all four layers)

Exit check

- Rows in each fact equal the Silver source after exclusions; no duplicate keys in any dimension or fact.
- Zero unexplained orphans.
- All ten KPI families return values that two independent queries agree on.

DAY 4 – Incremental Pipeline, Security, Snowpark Score and Recovery

Goal: Make the platform repeatable, secure and intelligent: incremental usage load, proven role security, an explainable risk score and a tested recovery procedure.

#	Task	Owner	Output
4.1	Create Stream on BRONZE usage table and a two-step Task graph (Task 1 MERGE into Silver usage; Task 2 refresh FACT_USAGE and MART_USAGE_SUMMARY). See C9.	M4	Stream/Task DDL
4.2	Test 1 – initial load: baseline counts (2,200 source rows less defects) in Silver, Gold and mart.	M4	Baseline evidence
4.3	Test 2 – second file: PUT batch 2, COPY, run the task. Prove new rows were inserted and corrected rows were updated (not duplicated).	M4, M2	Before/after counts and changed rows
4.4	Test 3 – idempotent replay: load the same batch 2 again (FORCE = TRUE) and run the task; counts and mart values must not change.	M4	Replay evidence
4.5	Test 4 – Gold-mart refresh: confirm MART_USAGE_SUMMARY reflects the corrected values.	M4	Mart before/after
4.6	Create roles NETWORK_ENGINEER_ROLE, SERVICE_ASSURANCE_ROLE, TELECOM_MANAGER_ROLE; grant warehouse, database, schema and object privileges; restrict Bronze to the engineering role (C10).	M4	RBAC script
4.7	Create masking policies for email, phone and address_line (and city/state if you decide they are address fields); apply to DIM_CUSTOMER and Silver customers.	M4	Masking script
4.8	Create SECURE VIEWs for all consumption objects; Streamlit and users read only these.	M4, M1	Secure view script
4.9	Validate each role with USE ROLE: run the same queries as each role and screenshot results (masked vs unmasked, allowed vs denied).	M4, M5	Role validation screenshots
4.10	Build the Snowpark risk score from Gold metrics (C11): component scores, total score, category, scoring timestamp into MART_NETWORK_RISK. Run twice to prove reproducibility.	M5	Snowpark code and output
4.11	Performance review: check query profiles for the 3 slowest queries, right-size warehouse, use result cache, avoid SELECT * in marts. Record before/after timings.	M3	Performance note
4.12	Recovery: demonstrate Time Travel query, CLONE at a past point, restoring a damaged table and UNDROP; write the rerun procedure for a failed load.	M3	Recovery evidence
4.13	Add monitoring queries (COPY_HISTORY, TASK_HISTORY, QUERY_HISTORY) and save them in the runbook folder.	M4	Monitoring script

Evidence to capture

- Stream, Task, TASK_HISTORY output and the four incremental test results
- Role-by-role screenshots (USE ROLE) showing masking and access limits
- Snowpark script, score table (two identical runs), recovery screenshots

Exit check

- Replay changes nothing; corrected records show the new values exactly once.

- No role other than engineering can query Bronze; customer contact and address are masked for roles without need.
- Risk score run twice gives identical component and total scores (only the timestamp differs).

DAY 5 – Streamlit, End-to-End Testing, UAT and Final Demo

Goal: Deliver a working application, prove everything with tests, close defects, hand over with a runbook and present confidently.

#	Task	Owner	Output
5.1	Build the Streamlit app (C12): Overview KPIs, Usage trends, Network reliability, Ticket SLA, Risk heat-list, Data-quality/exceptions page. It reads only secure views/marts.	M5, M1	Streamlit app
5.2	Streamlit-to-Gold reconciliation: for each KPI shown, a SQL query on Gold gives the same number. Capture side-by-side evidence.	M5, M3	Reconciliation screenshots
5.3	Run the complete test matrix (Part D1) from a fresh start: IDMC → Bronze → Silver → Gold → Task → Streamlit. Record pass/fail.	All	Test report
5.4	UAT: one person plays the business user and tries the five demo questions in Part D5 using only the dashboard; note any confusion.	M5, M1	UAT sign-off
5.5	Log defects with severity, owner and status; close all Critical and High defects; document any Low ones as known issues.	M5	Defect log
5.6	Write the runbook (D4 outline): how to run, rerun, monitor, recover, and who to contact.	M4, M3	Runbook
5.7	Finish the DQ report, reconciliation report, security evidence pack and architecture/mapping document.	M2, M1	Final reports
5.8	Prepare the final presentation (12–15 slides) and rehearse the demo twice; freeze code 2 hours before the demo.	M1, all	Presentation, demo script
5.9	Final demonstration (see D5), then tag the Git release.	All	Demo and release tag

Exit check

- All nine acceptance criteria in Part D3 are ticked with an evidence reference.
- A new person can follow the runbook and rerun the pipeline.
- Every deliverable in Part D4 exists in the repository.

PART C – HOW TO IMPLEMENT (LAYMAN GUIDANCE)

This part explains each step in simple words, then gives a starter you can adapt. Names such as TELCO_DB are suggestions. SQL shown is a **pattern**: adjust column lists to your real headers and value lists after profiling.

C1. Ten ideas explained simply

Term	Simple meaning	Everyday picture
Medallion (Bronze/Silver/Gold)	Three layers of increasing cleanliness.	River water → filtered water → bottled water.
IDMC	Informatica tool that cleans files and sorts rows before loading.	A quality-control conveyor belt with a reject bin.
Stage	A holding folder inside Snowflake for files.	A loading dock before the warehouse.
COPY INTO	Command that moves staged file rows into a table.	Unloading the truck onto shelves.
Grain	What exactly one row means in a table.	"One row = one usage transaction."
Dimension / Fact / Mart	Dimension = descriptive list (customer). Fact = events or measurements (usage). Mart = pre-summarised report table.	Phone book / call log / monthly bill summary.
Stream	Snowflake object that remembers which rows are new since you last looked.	A "new arrivals" tray.
Task	A scheduled job that runs SQL.	An alarm clock that starts a chore.
Idempotent	Running twice gives the same result as running once.	Pressing a lift button ten times still calls one lift.
Masking policy	Rule that hides a column's real value from some roles.	Showing ******1234 instead of the full phone number.

C2. Day 1 setup: Snowflake environment

Use ACCOUNTADMIN or SECURITYADMIN only where truly needed (creating roles, granting). Do everyday building with SYSADMIN.

```
USE ROLE SYSADMIN;
CREATE WAREHOUSE IF NOT EXISTS TELCO_WH
  WAREHOUSE_SIZE = 'XSMALL' AUTO_SUSPEND = 60 AUTO_RESUME = TRUE;
CREATE DATABASE IF NOT EXISTS TELCO_DB DATA_RETENTION_TIME_IN_DAYS = 1;
USE DATABASE TELCO_DB;
CREATE SCHEMA IF NOT EXISTS STG;          -- stages, file formats
CREATE SCHEMA IF NOT EXISTS BRONZE;       -- raw + metadata
CREATE SCHEMA IF NOT EXISTS SILVER;       -- typed, de-duplicated
CREATE SCHEMA IF NOT EXISTS GOLD;         -- dims, facts, marts, secure views
CREATE SCHEMA IF NOT EXISTS QUARANTINE;   -- rejects and duplicates
CREATE SCHEMA IF NOT EXISTS AUDIT;        -- batch, file, row-count logs
CREATE SCHEMA IF NOT EXISTS PIPELINE;     -- stream and tasks (same schema)
CREATE SCHEMA IF NOT EXISTS SECURITY;     -- masking policies
```

> **In plain words:** Tasks that depend on each other must sit in the same schema, which is why we keep a dedicated PIPELINE schema. Retention of 1 day is the safe default; if your account allows more, raise it for Time Travel demos.

C3. Day 1: profiling checklist (per file)

- **Shape:** row count equals the inventory (900, 1,400, 400, 2,200, 700, 400, 900) and column names/order equal the data dictionary.
- **Keys:** is the business key unique? Which keys repeat (the 3 duplicates)?

- **Nulls:** count per column; list the 8 controlled nulls and decide mandatory vs optional.
- **Dates:** list every pattern found (4 formats). Count rows per pattern.
- **Values:** SELECT DISTINCT for status, severity, priority, type and usage_type columns.
- **Ranges:** min/max/avg for numeric columns; find 1,000,000 MB and 500,000 customers.
- **Sequences:** end before start in subscriptions, incidents, tickets.
- **Relationships:** orphan counts for each join in A2.

Quick way to profile before IDMC: load each file once into a temporary all-VARCHAR table in a sandbox schema, then use SQL such as:

```
SELECT COUNT(*) total, COUNT(DISTINCT customer_id) distinct_keys,
       COUNT_IF(email IS NULL) null_email
FROM SANDBOX.CUSTOMERS_TMP;

-- date pattern detector
SELECT REGEXP_REPLACE(date_of_birth, '[0-9]', '9') AS pattern, COUNT(*) n
FROM SANDBOX.CUSTOMERS_TMP GROUP BY 1 ORDER BY n DESC;
```

C4. Day 2: IDMC cleansing and routing

Build the same shape of mapping seven times. Keep it simple and repeatable:

Step	What the mapping does	Typical IDMC building block
1	Read the source file; keep all columns in dictionary order.	Source transformation
2	Standardise: trim spaces, consistent upper/lower case for codes, convert every date format to ISO (YYYY-MM-DD or YYYY-MM-DD HH24:MI:SS).	Expression
3	Flag defects: null in optional column, end earlier than start, outlier threshold (e.g. data_volume_mb ≥ 1,000,000; impacted customers ≥ 500,000), duplicate key.	Expression + Sorter/Rank or Aggregator for duplicates
4	Route rows by flag: REJECT (impossible sequence) → DUPLICATE (second copy) → WARNING (optional null, outlier) → CLEAN (everything else, including standardised dates).	Router
5	Write four files per source: *_clean.csv, *_warning.csv, *_duplicate.csv, *_reject.csv, each with a REASON_CODE column on exceptions.	Targets
6	Link the seven mappings in one taskflow; log row counts for each target.	Taskflow

> **Watch out:** Route a row to **only one** target. If a row is both a duplicate and has an optional null, decide a priority order (Reject > Duplicate > Warning > Clean) and document it. Overlap is the most common reason counts fail to match.

Expected result if defects do not overlap: 3 rows in the reject files (one each in subscriptions, incidents, tickets), 3 in duplicate files, 8 + 2 in warning files (nulls and outliers), and the 4 mixed-format rows corrected and kept as clean. Combined, 6,900 source rows = 6,894 loaded + 3 duplicate + 3 reject.

Layman rules for the date formats: decide the one correct reading of each ambiguous string using other rows in the same column as evidence, and write the rule in the mapping notes. If you cannot decide, treat the row as a warning instead of guessing.

C5. Day 2: stages, file formats and Bronze loads

```
USE SCHEMA TELCO_DB.STG;
CREATE OR REPLACE FILE FORMAT FF_CSV TYPE = CSV FIELD_DELIMITER = ','
  SKIP_HEADER = 1 FIELD_OPTIONALLY_ENCLOSED_BY = '"' TRIM_SPACE = TRUE
  NULL_IF = ('', 'NULL', 'null') ERROR_ON_COLUMN_COUNT_MISMATCH = TRUE;
CREATE OR REPLACE FILE FORMAT FF_JSON TYPE = JSON STRIP_OUTER_ARRAY = TRUE;
CREATE OR REPLACE STAGE INT_TELCO_STAGE FILE_FORMAT = FF_CSV;
```

Upload from your laptop with SnowSQL (PUT only works from SnowSQL or a connector, not from the Snowsight SQL editor):

```
snowsql -a <account> -u <user> -r SYSADMIN -w TELCO_WH -d TELCO_DB
PUT file://C:/telco/clean/customers_clean.csv
    @STG.INT_TELCO_STAGE/customers/ AUTO_COMPRESS = TRUE;
LIST @STG.INT_TELCO_STAGE/customers/;
```

Bronze tables keep everything as text (so a bad value can never block a load) and add metadata so every row can be traced to its file and batch:

```
CREATE OR REPLACE TABLE BRONZE.CUSTOMERS_RAW (
  customer_id VARCHAR, customer_name VARCHAR, customer_type VARCHAR,
  date_of_birth VARCHAR, email VARCHAR, phone VARCHAR, address_line VARCHAR,
  city VARCHAR, state VARCHAR, region VARCHAR, customer_segment VARCHAR,
  registration_date VARCHAR, source_updated_ts VARCHAR,
  _source_file VARCHAR, _file_row_number NUMBER,
  _load_ts TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(), _batch_id VARCHAR);

COPY INTO BRONZE.CUSTOMERS_RAW
FROM (SELECT $1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,
             METADATA$FILENAME, METADATA$FILE_ROW_NUMBER,
             CURRENT_TIMESTAMP(), 'B001'
      FROM @STG.INT_TELCO_STAGE/customers/)
FILE_FORMAT = (FORMAT_NAME = 'STG.FF_CSV') ON_ERROR = 'ABORT_STATEMENT';
```

For the JSON file, store each event whole in a VARIANT column:

```
CREATE OR REPLACE TABLE BRONZE.NETWORK_EVENTS_RAW (
  raw VARIANT, _source_file VARCHAR, _file_row_number NUMBER,
  _load_ts TIMESTAMP_NTZ, _batch_id VARCHAR);
COPY INTO BRONZE.NETWORK_EVENTS_RAW
FROM (SELECT $1, METADATA$FILENAME, METADATA$FILE_ROW_NUMBER,
             CURRENT_TIMESTAMP(), 'B001'
      FROM @STG.INT_TELCO_STAGE/events/)
FILE_FORMAT = (FORMAT_NAME = 'STG.FF_JSON');
```

Audit after every load: insert into AUDIT a row per file with source rows, routed counts, rows loaded, rows rejected and the batch id. Use COPY INTO ... VALIDATION_MODE = RETURN_ERRORS as a dry run if you want to check a file first.

> **In plain words:** COPY INTO remembers which files it already loaded (for about 64 days) and skips them. That protects you from accidental double loads, and it is why the replay test in Day 4 uses FORCE = TRUE on purpose.

C6. Day 3: Silver layer

Silver = right data types, no duplicate keys, valid relationships. Use TRY_ functions: they return NULL instead of failing when a value is bad, so you can count and route problems.

```
CREATE OR REPLACE TABLE SILVER.CUSTOMERS AS
SELECT customer_id, customer_name, customer_type,
  TRY_TO_DATE(date_of_birth)              AS date_of_birth,
  email, phone, address_line, city, state, region, customer_segment,
  TRY_TO_DATE(registration_date)          AS registration_date,
  TRY_TO_TIMESTAMP_NTZ(source_updated_ts)  AS source_updated_ts,
  _source_file, _batch_id
FROM BRONZE.CUSTOMERS_RAW
QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id
        ORDER BY TRY_TO_TIMESTAMP_NTZ(source_updated_ts) DESC, _load_ts DESC) = 1;
```

Flatten the JSON events (nested objects stay as OBJECT, lists stay as ARRAY):

```
CREATE OR REPLACE TABLE SILVER.NETWORK_EVENTS AS
SELECT raw:event_id::VARCHAR                  AS event_id,
       raw:network_element_id::VARCHAR        AS network_element_id,
       TRY_TO_TIMESTAMP_NTZ(raw:event_timestamp::VARCHAR) AS event_timestamp,
       raw:event_type::VARCHAR AS event_type, raw:severity::VARCHAR AS severity,
       raw:metric_context::OBJECT AS metric_context,
       raw:alarm_context::OBJECT  AS alarm_context,
       raw:affected_services::ARRAY AS affected_services,
```

```
       raw:related_elements::ARRAY  AS related_elements,
       raw:geo_context::OBJECT AS geo_context,
       raw:event_status::VARCHAR AS event_status,
       raw:source_system::VARCHAR AS source_system
FROM BRONZE.NETWORK_EVENTS_RAW
QUALIFY ROW_NUMBER() OVER (PARTITION BY raw:event_id::VARCHAR
        ORDER BY _load_ts DESC) = 1;

-- explode services when you need one row per affected service
SELECT e.event_id, s.value::VARCHAR AS service
FROM SILVER.NETWORK_EVENTS e, LATERAL FLATTEN(input => e.affected_services) s;
```

Relationship check pattern (should return 0 or a documented exception list):

```
SELECT u.usage_transaction_id
FROM SILVER.USAGE_TRANSACTIONS u
LEFT JOIN SILVER.SUBSCRIPTIONS s ON u.subscription_id = s.subscription_id
WHERE s.subscription_id IS NULL;
```

Watch out: Do not guess keys inside the JSON. Open one record in Snowsight and look at the exact key names and nesting (for example whether severity appears at the top level or inside alarm_context) before writing the SELECT.

C7. Day 3: Gold model (grain and keys)

Table	Grain: one row per …	Key	Important columns / logic
DIM_DATE	calendar day	date_key (YYYYMMDD)	year, quarter, month, week, weekday, is_weekend. Generate with GENERATOR.
DIM_CUSTOMER	customer	customer_key (surrogate) + customer_id	Type, segment, region, city; PII columns carry masking policies.
DIM_SUBSCRIPTION	subscription	subscription_key + subscription_id	customer_key, service_type, status, start/end/contract dates, region; account_id, plan_id, device_id as plain attributes; is_active flag.
DIM_NETWORK_ELEMENT	network element	element_key + network_element_id	type, parent_element_id, hierarchy_level, hierarchy_path (recursive CTE), city, region, capacity, status.
FACT_USAGE	usage transaction	usage_transaction_id	subscription_key, customer_key, element_key, date_key, voice_minutes, sms_count, data_volume_mb, roaming_flag, charge_amount, is_outlier.
FACT_NETWORK_INCIDENT	incident	network_incident_id	element_key, start/end date_key, duration_minutes, severity, type, status, customers_impacted, is_outlier.
FACT_TROUBLE_TICKET	ticket	ticket_id	customer_key, subscription_key, incident key (nullable), resolution_minutes, sla_target_minutes, sla_met_flag, priority, status, team.
FACT_NETWORK_EVENT	event	event_id	element_key, date_key, event_type, severity, event_status, is_critical_open, affected_service_count, related_element_count.
MART_USAGE_SUMMARY	month × region × service/usage type	composite	total voice, SMS, data; active subscriptions; average data per subscription.
MART_NETWORK_RELIABILITY	month × element (or region)	composite	incident count, total and mean duration (MTTR), customers impacted, open critical alarms.
MART_TICKET_SLA	month × priority × team	composite	ticket count, average resolution minutes, SLA met %, breached count.
MART_NETWORK_RISK	network element × scoring run	element_key + scored_ts	component scores, total_risk_score, risk_category, scoring_ts, rule version.

```
-- Hierarchy path for DIM_NETWORK_ELEMENT
WITH RECURSIVE h AS (
```

```
  SELECT network_element_id, parent_element_id, 1 AS lvl,
         network_element_id::VARCHAR AS path
  FROM SILVER.NETWORK_ELEMENTS WHERE parent_element_id IS NULL
  UNION ALL
  SELECT c.network_element_id, c.parent_element_id, h.lvl + 1,
         h.path || ' > ' || c.network_element_id
  FROM SILVER.NETWORK_ELEMENTS c JOIN h ON c.parent_element_id = h.network_element_id)
SELECT * FROM h;
```

In plain words: Always state the grain in one sentence before writing a table. Most wrong KPIs come from joining two tables at different grains (for example joining tickets to usage and then summing, which multiplies rows). Aggregate each fact to a common grain first, then join.

C8. KPI catalogue (all §10 KPIs)

KPI	Formula (assumption in A4 where noted)	Source
Active subscriptions	COUNT of subscriptions with status = Active and (end date null or ≥ today)	DIM_SUBSCRIPTION
Voice usage	SUM(voice_minutes)	FACT_USAGE
SMS usage	SUM(sms_count)	FACT_USAGE
Data usage	SUM(data_volume_mb) (also show with outlier excluded)	FACT_USAGE
Avg data per subscription	SUM(data_volume_mb) / COUNT(DISTINCT subscription_id)	FACT_USAGE
Incident count	COUNT(network_incident_id)	FACT_NETWORK_INCIDENT
Incident duration	SUM and AVG of DATEDIFF(minute, start, end)	FACT_NETWORK_INCIDENT
Estimated customers impacted	SUM(estimated_customers_impacted) (flag the 500,000 outlier)	FACT_NETWORK_INCIDENT
Mean time to restore	AVG(duration_minutes) for resolved incidents	FACT_NETWORK_INCIDENT
Ticket volume	COUNT(ticket_id) by category/priority/month	FACT_TROUBLE_TICKET
Ticket resolution time	AVG(DATEDIFF(minute, create, resolve)) for resolved tickets	FACT_TROUBLE_TICKET
Ticket SLA compliance	COUNT(resolution_minutes ≤ sla_target_minutes) / COUNT(resolved tickets)	FACT_TROUBLE_TICKET
Open critical alarms	COUNT events severity = Critical and status not cleared	FACT_NETWORK_EVENT
High-risk network elements	COUNT elements with risk_category = High (latest run)	MART_NETWORK_RISK

C9. Day 4: incremental Stream and Task (usage only)

Idea: new rows land in Bronze; the Stream notices them; the Task merges them into Silver (insert new, update corrected) and then refreshes the Gold fact and mart. Because we MERGE on the business key and only update when the incoming record is newer, running the same file twice changes nothing.

```
USE SCHEMA TELCO_DB.PIPELINE;
CREATE OR REPLACE STREAM STRM_USAGE_RAW
  ON TABLE TELCO_DB.BRONZE.USAGE_RAW APPEND_ONLY = TRUE;

CREATE OR REPLACE TASK TSK_USAGE_MERGE
  WAREHOUSE = TELCO_WH SCHEDULE = '5 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('TELCO_DB.PIPELINE.STRM_USAGE_RAW')
AS
MERGE INTO TELCO_DB.SILVER.USAGE_TRANSACTIONS t
USING (
  SELECT usage_transaction_id, subscription_id, customer_id,
         TRY_TO_TIMESTAMP_NTZ(usage_ts) usage_ts, usage_type,
         TRY_TO_NUMBER(voice_minutes) voice_minutes,
         TRY_TO_NUMBER(sms_count) sms_count,
         TRY_TO_NUMBER(data_volume_mb, 18, 3) data_volume_mb,
         TRY_TO_BOOLEAN(roaming_flag) roaming_flag, network_element_id,
         TRY_TO_NUMBER(charge_amount, 18, 4) charge_amount,
```

```
        TRY_TO_TIMESTAMP_NTZ(source_updated_ts) source_updated_ts
  FROM TELCO_DB.PIPELINE.STRM_USAGE_RAW
  QUALIFY ROW_NUMBER() OVER (PARTITION BY usage_transaction_id
          ORDER BY TRY_TO_TIMESTAMP_NTZ(source_updated_ts) DESC) = 1) s
ON t.usage_transaction_id = s.usage_transaction_id
WHEN MATCHED AND s.source_updated_ts > t.source_updated_ts THEN UPDATE SET
  t.subscription_id = s.subscription_id, t.customer_id = s.customer_id,
  t.usage_ts = s.usage_ts, t.usage_type = s.usage_type,
  t.voice_minutes = s.voice_minutes, t.sms_count = s.sms_count,
  t.data_volume_mb = s.data_volume_mb, t.roaming_flag = s.roaming_flag,
  t.network_element_id = s.network_element_id,
  t.charge_amount = s.charge_amount, t.source_updated_ts = s.source_updated_ts
WHEN NOT MATCHED THEN INSERT
  (usage_transaction_id, subscription_id, customer_id, usage_ts, usage_type,
   voice_minutes, sms_count, data_volume_mb, roaming_flag,
   network_element_id, charge_amount, source_updated_ts)
VALUES (s.usage_transaction_id, s.subscription_id, s.customer_id, s.usage_ts,
   s.usage_type, s.voice_minutes, s.sms_count, s.data_volume_mb,
   s.roaming_flag, s.network_element_id, s.charge_amount, s.source_updated_ts);

CREATE OR REPLACE TASK TSK_USAGE_GOLD_REFRESH
  WAREHOUSE = TELCO_WH AFTER TSK_USAGE_MERGE
AS CALL TELCO_DB.GOLD.SP_REFRESH_USAGE();    -- MERGE FACT_USAGE, rebuild mart

ALTER TASK TSK_USAGE_GOLD_REFRESH RESUME;    -- resume child first
ALTER TASK TSK_USAGE_MERGE RESUME;
EXECUTE TASK TSK_USAGE_MERGE;                -- manual run for demos
```

Watch out: Your role needs EXECUTE TASK on the account and ownership/usage on the objects. Create tasks suspended, test with EXECUTE TASK, then resume. Suspend the schedule after the demo so it does not consume credits. SP_REFRESH_USAGE is a small SQL stored procedure that you write: it merges new rows into FACT_USAGE and rebuilds MART_USAGE_SUMMARY.

The four tests to run and screenshot:

Test	Do this	Expected result
1. Initial load	Load usage_transactions clean/warning files; run the task.	Silver and FACT_USAGE rows = source 2,200 less routed defects; mart totals match.
2. Second file	PUT and COPY batch 2 (new + corrected records); run the task.	Row count rises by the number of new IDs only; corrected IDs show new values; no duplicate IDs.
3. Idempotent replay	COPY batch 2 again with FORCE = TRUE; run the task.	Silver, fact and mart values identical to after Test 2.
4. Mart refresh	Compare MART_USAGE_SUMMARY before and after Test 2.	Mart reflects the new and corrected values.

C10. Day 4: security (roles, masking, secure views)

Object	NETWORK_ENGINEER_ROLE	SERVICE_ASSURANCE_ROLE	TELECOM_MANAGER_ROLE
BRONZE / STG / QUARANTINE	Read	No access	No access
SILVER	Read	No access (assumption)	No access
GOLD secure views and marts	Read	Read	Read
Customer email, phone, address	Masked	Unmasked (needs to contact customers)	Masked
Snowpark risk table	Read	Read	Read

```
USE ROLE SECURITYADMIN;
CREATE ROLE IF NOT EXISTS NETWORK_ENGINEER_ROLE;
CREATE ROLE IF NOT EXISTS SERVICE_ASSURANCE_ROLE;
CREATE ROLE IF NOT EXISTS TELECOM_MANAGER_ROLE;
GRANT ROLE NETWORK_ENGINEER_ROLE TO ROLE SYSADMIN;  -- keep admin tree tidy
GRANT ROLE SERVICE_ASSURANCE_ROLE TO ROLE SYSADMIN;
GRANT ROLE TELECOM_MANAGER_ROLE TO ROLE SYSADMIN;
```

```
USE ROLE SYSADMIN;
GRANT USAGE ON WAREHOUSE TELCO_WH TO ROLE SERVICE_ASSURANCE_ROLE;
GRANT USAGE ON DATABASE TELCO_DB TO ROLE SERVICE_ASSURANCE_ROLE;
GRANT USAGE ON SCHEMA TELCO_DB.GOLD TO ROLE SERVICE_ASSURANCE_ROLE;
-- engineering only for Bronze
GRANT USAGE ON SCHEMA TELCO_DB.BRONZE TO ROLE NETWORK_ENGINEER_ROLE;
GRANT SELECT ON ALL TABLES IN SCHEMA TELCO_DB.BRONZE
  TO ROLE NETWORK_ENGINEER_ROLE;

CREATE OR REPLACE MASKING POLICY SECURITY.MP_CONTACT AS (v STRING)
RETURNS STRING ->
  CASE WHEN CURRENT_ROLE() IN ('SERVICE_ASSURANCE_ROLE') THEN v
       WHEN v IS NULL THEN NULL ELSE '*** MASKED ***' END;
ALTER TABLE GOLD.DIM_CUSTOMER MODIFY COLUMN email
  SET MASKING POLICY SECURITY.MP_CONTACT;
-- repeat for phone and address_line

CREATE OR REPLACE SECURE VIEW GOLD.VW_CUSTOMER AS
SELECT customer_key, customer_id, customer_type, customer_segment,
       city, state, region, email, phone, address_line
FROM GOLD.DIM_CUSTOMER;
GRANT SELECT ON VIEW GOLD.VW_CUSTOMER TO ROLE SERVICE_ASSURANCE_ROLE;

-- validate
USE ROLE SERVICE_ASSURANCE_ROLE;  SELECT email FROM GOLD.VW_CUSTOMER LIMIT 3;
USE ROLE TELECOM_MANAGER_ROLE;    SELECT email FROM GOLD.VW_CUSTOMER LIMIT 3;
USE ROLE TELECOM_MANAGER_ROLE;    SELECT * FROM BRONZE.CUSTOMERS_RAW;  -- must fail
```

Also grant each role to your own test user so you can switch with USE ROLE. A role may need USAGE on the secure view's schema and SELECT on the view. Screenshot every success **and** every expected denial.

> **In plain words:** Roles are job titles with permission lists. Masking is a rule that says "the same column shows different values to different job titles". A secure view is a window that hides how the data behind it is built.

C11. Day 4: Snowpark network-risk score

Create a simple, explainable rule-based score. Anyone should be able to recompute it with a calculator. No machine learning is needed.

Component	Measure (per element)	Score 0–100 rule	Weight
Incident frequency	Incident count	Scale against the highest element count	25%
Severity	Severity-weighted incident points (e.g. Low 1, Medium 2, High 3, Critical 4)	Scale against the maximum	25%
Restore time	Average incident duration (minutes)	Scale, with extreme values capped (for example at the 95th percentile)	20%
Open critical alarms	Count of open critical events	Scale against the maximum	20%
Customer-experience impact	Ticket count and SLA breaches linked to the element's incidents	Scale against the maximum	10%

Total score = sum of (component score × weight). **Category:** Low if total < 40, Medium if 40 to 69, High if 70 or above (adjust after seeing the real distribution, then freeze and document the thresholds). Cap the 500,000-customer incident so one outlier cannot dominate the score.

```
from snowflake.snowpark import Session
from snowflake.snowpark.functions import col, lit, when, current_timestamp, least

def scale(c, mx):        # 0-100, never above 100
    return least(lit(100.0), (c / lit(mx)) * lit(100.0))

def run(session: Session):
    inc = session.table("GOLD.MART_NETWORK_RELIABILITY").group_by("element_key") \
        .agg({"incident_count": "sum", "severity_points": "sum",
              "avg_duration_min": "avg", "open_critical_alarms": "sum",
```

```
            "ticket_impact": "sum"}) \
        .to_df("element_key", "inc", "sev", "dur", "alm", "tkt")
    mx = inc.agg({"inc": "max", "sev": "max", "dur": "max",
                  "alm": "max", "tkt": "max"}).collect()[0]   # fixed maxima
    s = inc.with_columns(
        ["inc_score", "sev_score", "dur_score", "alm_score", "tkt_score"],
        [scale(col("inc"), mx[0]), scale(col("sev"), mx[1]),
         scale(col("dur"), mx[2]), scale(col("alm"), mx[3]),
         scale(col("tkt"), mx[4])])
    s = s.with_column("total_risk_score",
        col("inc_score")*0.25 + col("sev_score")*0.25 + col("dur_score")*0.20
        + col("alm_score")*0.20 + col("tkt_score")*0.10)
    s = s.with_column("risk_category",
        when(col("total_risk_score") >= 70, lit("High"))
        .when(col("total_risk_score") >= 40, lit("Medium")).otherwise(lit("Low")))
    s = s.with_column("scoring_ts", current_timestamp())
    s.write.mode("overwrite").save_as_table("GOLD.MART_NETWORK_RISK")
    return s
```

Watch out: The code above is a pattern: column names such as severity_points and ticket_impact must exist in your mart (add them on Day 3 or derive them in the script). Test the aggregate-result indexing in your Snowpark version, then validate by recomputing 3 elements by hand in a spreadsheet and matching the stored values. Run twice: component and total scores must be identical; only scoring_ts changes.

C12. Day 5: Streamlit application

Run it inside Snowflake (Streamlit in Snowsight) so it uses your session and reads only secure views and marts.

Page	Contents	Source
Overview	KPI cards: active subscriptions, total data/voice/SMS, incidents, MTTR, SLA %, open critical alarms, high-risk elements	Marts + risk table
Usage	Trend by month and region; split voice/SMS/data; roaming share	MART_USAGE_SUMMARY
Network reliability	Incidents by severity and region; duration trend; customers impacted; element drill-down	MART_NETWORK_RELIABILITY
Ticket SLA	Volume, resolution time, SLA % by priority and team	MART_TICKET_SLA
Network risk	Ranked list with component scores and category colour; filter by region	MART_NETWORK_RISK
Data quality / exceptions	Counts routed clean/warning/duplicate/reject; reconciliation summary	AUDIT views

```
import streamlit as st
from snowflake.snowpark.context import get_active_session
session = get_active_session()

st.title("Telecom Service Assurance")
region = st.sidebar.selectbox("Region",
    ["All"] + [r[0] for r in session.sql(
        "SELECT DISTINCT region FROM GOLD.VW_NETWORK_ELEMENT").collect()])

kpi = session.sql("""
  SELECT SUM(incident_count) inc, AVG(mttr_min) mttr
  FROM GOLD.VW_NETWORK_RELIABILITY
  WHERE (:r = 'All' OR region = :r)""".replace(":r", f"'{region}'")).to_pandas()
c1, c2 = st.columns(2)
c1.metric("Incidents", int(kpi.INC[0]))
c2.metric("MTTR (min)", round(float(kpi.MTTR[0]), 1))

risk = session.table("GOLD.VW_NETWORK_RISK").to_pandas()
st.dataframe(risk.sort_values("TOTAL_RISK_SCORE", ascending=False))
st.bar_chart(risk.groupby("RISK_CATEGORY").size())
```

Reconciliation rule: each number on screen must be reproducible by a plain SQL query on Gold. Keep a table of "dashboard number | SQL | result | match?" for the final evidence.

> **Watch out:** The region filter above builds SQL from a dropdown value, which is acceptable here because the options come from the database, but never build SQL from free-typed text. Prefer filtering the pandas DataFrame after loading when data is small.

C13. Day 5: end-to-end test and reconciliation approach

- **Reconcile counts first, values second.** Counts: source = clean + warning + duplicate + reject; Bronze = clean + warning; Silver = Bronze minus Silver-level exclusions; Gold facts = Silver. Values: SUM(data_volume_mb), SUM(charge_amount), SUM(estimated_customers_impacted).
- **Test from a clean state.** Drop and rebuild everything once from the repository scripts to prove the runbook works.
- **Keep evidence while you work.** Take screenshots with the role, timestamp and query visible.
- **Explain every difference.** An unexplained count difference of even 1 fails the acceptance criteria.

C14. Day 4: recovery, rerun and monitoring

```
-- 1. See the table as it was 10 minutes ago
SELECT COUNT(*) FROM GOLD.FACT_USAGE AT (OFFSET => -60*10);

-- 2. Safe copy from the past (clone), then swap in if needed
CREATE OR REPLACE TABLE GOLD.FACT_USAGE_RESTORE
  CLONE GOLD.FACT_USAGE AT (OFFSET => -60*10);
ALTER TABLE GOLD.FACT_USAGE SWAP WITH GOLD.FACT_USAGE_RESTORE;

-- 3. Undo an accidental drop
DROP TABLE GOLD.MART_TICKET_SLA;
UNDROP TABLE GOLD.MART_TICKET_SLA;

-- monitoring
SELECT * FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
  TASK_NAME => 'TSK_USAGE_MERGE')) ORDER BY SCHEDULED_TIME DESC;
SELECT * FROM TABLE(INFORMATION_SCHEMA.COPY_HISTORY(
  TABLE_NAME => 'USAGE_RAW', START_TIME => DATEADD(hour, -24, CURRENT_TIMESTAMP())));
```

Rerun procedure for a failed load (write this in the runbook): (1) check TASK_HISTORY/COPY_HISTORY for the error; (2) fix the file or rule; (3) if partial data was loaded, restore the table using Time Travel or clone, or delete the rows of that _batch_id; (4) reload with a new batch id; (5) rerun the task; (6) re-run reconciliation and record it in the audit log.

C15. Day 5: runbook outline

- Purpose, architecture diagram and object inventory
- Prerequisites: accounts, roles, warehouse, tools, secrets handling
- Daily run steps: IDMC taskflow → PUT → COPY → tasks → checks
- Monitoring queries and what "healthy" looks like
- Rerun, rollback and recovery procedures (C14)
- Security: role matrix, how to add a user, how masking works
- Troubleshooting table: symptom, likely cause, fix
- Contacts, change log and known issues

PART D – TRACKERS AND CHECKLISTS

D1. Test matrix (§15)

ID	Area	Test	Expected	Result
T01	Load	CSV/JSON load; header names and order vs dictionary for all 7 files	All columns present in correct order	
T02	Datatype	Casting and mandatory-field checks in Silver	No unexpected NULLs after TRY_ casts	
T03	DQ routing	8 nulls, 3 duplicates, 3 anomalies, 2 outliers, 4 formats	Each routed exactly as §8.3	
T04	Relationships	Orphan queries for every join in A2	0 orphans or documented exceptions	
T05	Sequences	End ≥ start for subscriptions, incidents, tickets	0 violations in Silver/Gold	
T06	Reconciliation	Source → Bronze → Silver → Gold counts and sums	All differences explained, none unexplained	
T07	KPI	Each §10 KPI computed two ways	Both ways agree	
T08	Incremental	Initial load, second file, replay, mart refresh	See C9 expected results	
T09	Security	USE ROLE tests for 3 roles incl. denied Bronze access	Matches access matrix	
T10	Masking	Email, phone, address as each role	Masked/unmasked as designed	
T11	Snowpark	Run score twice; hand-check 3 elements	Identical scores; hand check matches	
T12	Streamlit	Each dashboard number vs Gold SQL	Exact match	
T13	Recovery	Time Travel query, clone, UNDROP, rerun procedure	Data restored; rerun works	
T14	Clean rebuild	Rebuild from scripts in a fresh schema	Same counts as original	

D2. Reconciliation template

Fill one row per file. Source counts come from the dataset inventory; the others are your results. Expected values assume defects do not overlap (confirm on Day 1).

File	Source	Clean	Warning	Duplicate	Reject	Bronze	Silver	Gold
customers	900							
subscriptions	1,400							
network_elements	400							
usage_transactions	2,200							
trouble_tickets	700							
network_incidents	400							
network_events	900							
Total	6,900		8 nulls + 2 outliers	3	3			

Check: Source = Clean + Warning + Duplicate + Reject. Bronze = Clean + Warning. Silver and Gold fact rows follow from Bronze after dimension-level exclusions.

D3. Acceptance criteria tracker (§17)

#	Criterion	Evidence	Done
1	All seven files and 6,900 records processed	Reconciliation report, audit log	☐
2	Every documented column present in correct order	Header test T01, Bronze DDL vs dictionary	☐
3	Controlled DQ records detected and routed exactly	DQ register, IDMC outputs, DQ report	☐
4	Relationships, grain, KPIs and reconciliations correct	T04–T07 results	☐
5	Incremental processing is idempotent	T08 before/after counts	☐
6	Roles and masking demonstrated	USE ROLE screenshots	☐
7	Snowpark score reproducible	Two identical runs, hand check	☐
8	Streamlit reconciles with Gold	Dashboard-vs-SQL table	☐
9	Recovery and rerun procedures work	Time Travel/clone evidence, runbook test	☐

D4. Final deliverables checklist (§16)

Deliverable	Content	Owner	Done
Architecture and mapping	Diagram, grain/keys, source-to-target mapping	M1	☐
IDMC mappings and taskflow evidence	7 mappings, taskflow run, routed files	M2	☐
Snowflake scripts and objects	Numbered SQL scripts in Git	M3	☐
Stream and Task	DDL, TASK_HISTORY, four test results	M4	☐
Snowpark code and output	Script, score table, reproducibility proof	M5	☐
Streamlit application	App code and running demo	M5	☐
DQ, reconciliation and test reports	D1, D2 filled and signed	M2, M5	☐
Security evidence	Role matrix, masking and denial screenshots	M4	☐
Runbook and final presentation	C15 outline; 12–15 slides	M4, M1	☐

D5. Final demo script (about 15 minutes)

- **1. Problem and architecture (2 min):** the eight layers and the seven files.
- **2. Data quality (2 min):** show the DQ register versus routed outputs: 8 / 3 / 3 / 2 / 4.
- **3. Lineage (2 min):** one customer traced from source file to Bronze, Silver and Gold.
- **4. Incremental (3 min):** run batch 2, show corrected rows, replay and unchanged counts.
- **5. Security (2 min):** switch roles with USE ROLE; show masking and the denied Bronze query.
- **6. Risk and dashboard (3 min):** high-risk element, its component scores and the matching Streamlit page.
- **7. Recovery and close (1 min):** Time Travel restore and the acceptance checklist.

Five questions the business user should answer from the dashboard alone: (1) Which region has the highest data usage? (2) Which elements are High risk and why? (3) What is the SLA compliance by priority? (4) How long do critical incidents take to restore? (5) How many open critical alarms exist right now?

D6. Common pitfalls and how to avoid them

Pitfall	Why it hurts	Prevention
Skipping profiling	You cannot prove exact DQ counts later	Finish the DQ register on Day 1
Rows matching more than one route	Counts will not add up to 6,900	Define priority order Reject > Duplicate > Warning > Clean
Guessing ambiguous dates (03/04/2026)	Silent wrong data	Use evidence from the column; else warn
Typed Bronze columns	Load fails on one bad value	Keep Bronze VARCHAR; type in Silver
Joining facts at different grains	Inflated KPIs	Aggregate to a common grain before joining
Forgetting outliers in averages	Skewed KPIs and risk score	Flag, show with/without, cap in scoring
Tasks left running	Wasted credits	Suspend tasks and warehouse after demos
Using ACCOUNTADMIN for daily work	Breaks the least-privilege requirement	Use SYSADMIN; admin roles only to create roles/grants
Evidence gathered at the end	Missing or unreadable proof	Screenshot as you go into /evidence/dayN
Building out-of-scope items	Time lost (billing, plan master, real-time)	Re-read §3.2 whenever tempted